

Accenture ASE / AASE / ATE

20-Day HTML + CSS + JavaScript Coding Practice Plan

Daily practice: Aug 30 – Sep 18 • Exam target: Sep 20

How to use this PDF

Do exactly one task per day. First read the 'What to make' section, then build it yourself. Do not start by copying a YouTube/tutorial solution. Spend roughly 60–90 minutes building, then 20–30 minutes fixing and improving it. The goal is not to make a beautiful project; the goal is to become fast at HTML structure, CSS layout, JavaScript DOM manipulation, events, conditions, arrays, and basic browser APIs.

Day	Date	Task	Main skill
1	Aug 30	Counter App	DOM manipulation, events
2	Aug 31	Calculator	events, strings, numbers, conditions
3	Sep 1	To-Do List	createElement, appendChild, event handling
4	Sep 2	Character Counter	input events, string length
5	Sep 3	Temperature Converter	arithmetic, parsing, validation
6	Sep 4	BMI Calculator	calculations, conditions, DOM
7	Sep 5	Digital Clock	Date, setInterval, formatting
8	Sep 6	Password Show/Hide	DOM properties, events
9	Sep 7	Form Validation	forms, preventDefault, validation
10	Sep 8	Number Guessing Game	Math.random, conditions, state
11	Sep 9	Rock Paper Scissors	arrays, random, conditions
12	Sep 10	Quiz App	arrays/objects, DOM, state
13	Sep 11	Search Filter	filtering, string methods, input event
14	Sep 12	Modal Popup	classList, events, UI state
15	Sep 13	Accordion	classList, DOM traversal
16	Sep 14	Image Slider	arrays, indexes, DOM updates
17	Sep 15	Expense Tracker	arrays/objects, calculations, DOM
18	Sep 16	Stopwatch	timers, state, events
19	Sep 17	Dark/Light Mode	CSS classes, classList, UI state
20	Sep 18	Final Mini Project: Student Task Manager	DOM, events, arrays, validation, timers/state

Day 1 — Aug 30: Counter App

What to make: Build a card with a number and –, Reset, + buttons.

What the task teaches: Learn DOM selection, click events, and changing text. The number should increase by 1, decrease by 1, and reset to 0.

Build requirements: HTML: heading, count display, 3 buttons. CSS: centered card, spacing, button styles. JS: counter variable + event listeners +.textContent.

Skills: Core: DOM manipulation, events

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 2 — Aug 31: Calculator

What to make: Build a simple calculator for +, −, ×, ÷ with a display.

What the task teaches: The user enters numbers/operators and presses = to see the result. Add Clear.

Build requirements: Create calculator buttons and display. Use JS to collect input, perform operations, and update the display.

Skills: Core: events, strings, numbers, conditions

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 3 — Sep 1: To-Do List

What to make: Build an input, Add button, and task list.

What the task teaches: A user should type a task, add it to the list, mark it complete, and delete it.

Build requirements: Create task elements dynamically with JS. Handle empty input and button clicks.

Skills: Core: createElement, appendChild, event handling

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 4 — Sep 2: Character Counter

What to make: Build a textarea with a live character count.

What the task teaches: As the user types, show how many characters have been entered. Optionally set a limit.

Build requirements: Listen to input events and update the count using value.length.

Skills: Core: input events, string length

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 5 — Sep 3: Temperature Converter

What to make: Build Celsius/Fahrenheit input fields or a selector-based converter.

What the task teaches: Enter a temperature and convert it instantly or when Convert is clicked.

Build requirements: Use the conversion formulas, validate numeric input, and display a rounded answer.

Skills: Core: arithmetic, parsing, validation

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 6 — Sep 4: BMI Calculator

What to make: Build inputs for weight and height with a Calculate button.

What the task teaches: Calculate BMI and show a category such as underweight, normal, overweight, or obese.

Build requirements: Read input values, calculate BMI, use if/else conditions, and display the result.

Skills: Core: calculations, conditions, DOM

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 7 — Sep 5: Digital Clock

What to make: Build a live clock showing hours, minutes, and seconds.

What the task teaches: The displayed time should update every second.

Build requirements: Create a Date object, format the time, and use setInterval to refresh the display.

Skills: Core: Date, setInterval, formatting

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 8 — Sep 6: Password Show/Hide

What to make: Build a password field and a Show/Hide button.

What the task teaches: Clicking the button should switch the field between password and text types.

Build requirements: Use input.type to toggle between 'password' and 'text' and change the button label.

Skills: Core: DOM properties, events

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 9 — Sep 7: Form Validation

What to make: Build a registration form with name, email, password, and submit.

What the task teaches: Prevent submission when fields are invalid and show useful error messages.

Build requirements: Validate empty fields, email format, password length, and matching confirmation if included.

Skills: Core: forms, preventDefault, validation

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 10 — Sep 8: Number Guessing Game

What to make: Generate a random number and let the user guess it.

What the task teaches: Tell the user whether the guess is too high, too low, or correct. Track attempts.

Build requirements: Generate a random number with Math.random, compare guesses, and update status.

Skills: Core: Math.random, conditions, state

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 11 — Sep 9: Rock Paper Scissors

What to make: Build three choice buttons and a result area.

What the task teaches: The computer randomly chooses Rock, Paper, or Scissors. Show winner/loser/draw and score.

Build requirements: Use an array for choices, random selection, and condition logic for the three outcomes.

Skills: Core: arrays, random, conditions

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 12 — Sep 10: Quiz App

What to make: Build multiple-choice questions with Next and score display.

What the task teaches: Show one question at a time, accept an answer, and show the final score.

Build requirements: Store questions in an array of objects and update the UI as the user moves through the quiz.

Skills: Core: arrays/objects, DOM, state

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 13 — Sep 11: Search Filter

What to make: Build a search box and a list of items.

What the task teaches: Typing in the search box should immediately show matching items and hide non-matching ones.

Build requirements: Listen for input, compare lowercase text, and update visibility.

Skills: Core: filtering, string methods, input event

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 14 — Sep 12: Modal Popup

What to make: Build a button that opens a modal with a close button.

What the task teaches: Click Open to show the modal, Close to hide it, and optionally close by clicking outside.

Build requirements: Toggle a CSS class or display property from JS.

Skills: Core: classList, events, UI state

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 15 — Sep 13: Accordion

What to make: Build several question/answer sections.

What the task teaches: Click a question to expand its answer and click again to collapse it.

Build requirements: Use event listeners and class toggling. Optionally allow only one section open at a time.

Skills: Core: classList, DOM traversal

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 16 — Sep 14: Image Slider

What to make: Build an image area with Previous and Next buttons.

What the task teaches: Clicking Next/Previous should change the displayed image and wrap around at the ends.

Build requirements: Store image URLs in an array and maintain a current index.

Skills: Core: arrays, indexes, DOM updates

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 17 — Sep 15: Expense Tracker

What to make: Build inputs for expense name and amount plus an expense list and total.

What the task teaches: Add expenses, delete them, and keep the total updated.

Build requirements: Store expenses in an array and recalculate the total after add/delete operations.

Skills: Core: arrays/objects, calculations, DOM

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 18 — Sep 16: Stopwatch

What to make: Build Start, Stop, and Reset controls with a time display.

What the task teaches: Start counting, pause without resetting, and reset to zero.

Build requirements: Use setInterval, clearInterval, and state variables to control the timer.

Skills: Core: timers, state, events

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 19 — Sep 17: Dark/Light Mode

What to make: Build a page with a theme toggle button.

What the task teaches: Clicking the toggle should switch the whole interface between light and dark themes.

Build requirements: Use CSS variables or theme classes and toggle them with classList.

Skills: Core: CSS classes, classList, UI state

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Day 20 — Sep 18: Final Mini Project: Student Task Manager

What to make: Combine a form, task list, filters, counter, and clean responsive UI.

What the task teaches: This is your mock coding-round task. Build it mostly without copying a tutorial. Add tasks, mark complete, delete, filter All/Active/Completed, and show counts.

Build requirements: Reuse patterns from previous days. Focus on structure, readable JS, DOM updates, validation, and finishing within a time limit.

Skills: Revision: DOM, events, arrays, validation, timers/state

Done means: It works from a fresh page, buttons/inputs behave correctly, obvious edge cases are handled, and you can explain your JavaScript line-by-line at a basic level.

Final 3-Day Strategy

Sep 16 — Stopwatch

Focus on timers and state. Understand why `setInterval` alone is not enough and why you need `clearInterval`.

Sep 17 — Dark/Light Mode

Focus on `classList` and separating styling from logic. Your JS should mainly change state/classes, while CSS controls appearance.

Sep 18 — Student Task Manager

Treat this as a mock coding round. Set a timer. Build from scratch. Avoid searching for the exact solution. If you get stuck, identify the specific concept you don't understand and fix that.

Minimum concepts you should be able to explain before Sep 20

- `querySelector` / `querySelectorAll`
- `textContent` / `innerHTML` / `value`
- `addEventListener` and common events: `click`, `input`, `submit`
- `preventDefault()`
- `classList.add` / `remove` / `toggle`
- `createElement`, `appendChild`, `remove`
- `if` / `else` and comparison operators
- arrays, objects, `push`, `filter`, `find`, `map` (basic use)
- `Math.random` and `Math.floor`
- `Date` and `setInterval` / `clearInterval`
- `parseInt` / `parseFloat` / `Number`

Brutal rule: If you can only reproduce these projects after watching a tutorial, you are not ready. The actual preparation is the ability to receive a small requirement, decide the HTML structure, style it quickly, write the JS logic, debug it, and finish under time pressure.